
MiniGames App

TP note – Seance 2 sur 3

Developpement Android avec Jetpack Compose

Theme de la seance :

Architecture & Navigation

ViewModel, NavHost, Mot cache – Trouve le mot dans la grille

Formation : CNAM – 1ere annee

Travail : En binome

Duree : 4h

Auteur : Jordan Hiertz

hiertzjordan@gmail.com

Ce sujet est le deuxieme volet d'un TP progressif sur 3 seances.
Vous repartez du code rendu lors de la seance 1.

Vue d'ensemble

Vous allez faire évoluer votre **MiniGames App** en introduisant deux concepts fondamentaux du développement Android professionnel : la séparation des responsabilités avec **ViewModel** et la navigation multi-écrans avec **NavHost**.

Progression sur 3 séances

Séance 1	Kotlin + Compose	Ecran d'accueil, jeu de réaction
Séance 2	Architecture + Navigation	ViewModel, NavHost, Mot cache
Séance 3	Base de données	Room, scores, leaderboard

A la fin de cette séance, votre application disposera d'une navigation propre entre trois écrans et d'un nouveau jeu : une **grille de lettres** où le joueur doit retrouver un mot caché parmi 9 lettres mélangées.

Objectifs pédagogiques

1. Comprendre le rôle du **ViewModel** et pourquoi il sépare logique et affichage.
2. Refactoriser le jeu de réaction pour y déplacer toute la logique métier.
3. Mettre en place une navigation multi-écrans avec **NavHost** et **NavController**.
4. Implémenter un nouveau jeu avec une logique d'état plus riche.

Modalités de rendu

Date limite : vendredi 12 juin 2025, 23h59.

Les projets seront clonés le samedi matin, aucun commit après cette date ne sera pris en compte.

Dépôt du code

Déposez votre projet sur un dépôt Git accessible (GitHub, GitLab...) et **envoyez le lien par Discord ou par mail** avant la date limite. Le dépôt peut être privé ; dans ce cas, pensez à m'y donner accès.

Branche à livrer

Pécisez dans le README la **branche contenant la version finale** à évaluer (main, develop ou autre). C'est cette branche qui sera clonée et testée.

README

Le README doit contenir à minima :

1. **Nom(s)** du binôme ou de la personne.
2. **Branche** à utiliser pour tester le projet.
3. **Ce qui fonctionne** : listez les fonctionnalités implémentées et opérationnelles.
4. **Ce qui ne fonctionne pas** : et pourquoi.

1. Mise en route

1.1. Nouvelles dependances

Trois ajouts sont nécessaires. Dans `gradle/libs.versions.toml` :

```
[versions]
# ...
lifecycleViewmodelCompose = "2.10.0"
navigationCompose          = "2.9.0"

[libraries]
# ...
lifecycle-viewmodel-compose = {
    group = "androidx.lifecycle",
    name  = "lifecycle-viewmodel-compose",
    version.ref = "lifecycleViewmodelCompose"
}
navigation-compose = {
    group = "androidx.navigation",
    name  = "navigation-compose",
    version.ref = "navigationCompose"
}

[plugins]
# ...
kotlin-serialization = {
    id = "org.jetbrains.kotlin.plugin.serialization",
    version.ref = "kotlin"
}
```

Dans `build.gradle.kts` (module), ajoutez le plugin en haut du fichier et la dépendance dans le bloc `dependencies` :

```
plugins {
    // ...
    alias(libs.plugins.kotlin.serialization)
}

dependencies {
    // ...
    implementation(libs.lifecycle.viewmodel.compose)
    implementation(libs.navigation.compose)
}
```

Synchronisez le projet après cette modification (*Sync Now*).

1.2. Structure des packages

Ajoutez les nouveaux fichiers à la structure existante :

```
com.votregroupe.minigames/
|-- ui/
|   |-- home/
|   |   '-- HomeScreen.kt          (modifier)
|   |-- reaction/
|   |   |-- ReactionScreen.kt      (modifier)
|   |   '-- ReactionViewModel.kt   (nouveau)
|   '-- wordgame/
|       |-- WordGameScreen.kt      (nouveau)
|       '-- WordGameViewModel.kt   (nouveau)
```

```
|-- Navigation.kt          (nouveau)  
'-- MainActivity.kt       (modifier)
```

2. Architecture : introduction au ViewModel

Dans la seance 1, toute la logique du jeu (timer, etats, calculs) etait directement dans le composable `ReactionScreen`. Cette approche a deux inconvenients :

- L'etat est **perdu** si l'ecran est recompose de facon inattendue (rotation du telephone, par exemple).
- Le composable devient difficile a lire et a tester.

Le **ViewModel** est une classe qui *survit* aux recompositions et aux changements de configuration. Il detient l'etat et la logique metier ; le composable se contente de l'afficher et d'appeler ses methodes.

```
// Declaration du ViewModel
class MonViewModel : ViewModel() {
    private val _score = MutableStateFlow(0)
    val score: StateFlow<Int> = _score.asStateFlow()

    fun incrementer() { _score.value++ }
}

// Utilisation dans un composable
@Composable
fun MonEcran(viewModel: MonViewModel = viewModel()) {
    val score by viewModel.score.collectAsState()
    Text("Score : $score")
    Button(onClick = { viewModel.incrementer() }) {
        Text("+ 1")
    }
}
```

`StateFlow` est la solution privilegiee dans un `ViewModel` : il ne depend pas de `Compose` et reste testable independamment. `collectAsState()` fait le lien avec la recomposition – l'UI se met a jour a chaque emission. `viewModel()` cree l'instance la premiere fois et la reutilise ensuite.

2.1. Refactorisation de `ReactionViewModel`

Creez `ReactionViewModel.kt` et deplacez-y tout ce qui se trouvait dans `ReactionScreen` : les etats (quelle que soit la forme que vous leur avez donnee – variables separees, data class...) ainsi que la logique du timer. Le `ViewModel` doit au minimum exposer ces trois methodes :

```
class ReactionViewModel : ViewModel() {

    // Vos etats ici (StateFlow, data class, etc.)

    fun startGame() { ... } // genere une partie et lance le timer
    fun stopTimer() { ... } // stoppe et calcule l'ecart
    fun reset()      { ... } // retour a l'etat initial
}
```

Le composable `ReactionScreen` doit etre allège : il ne fait plus que lire les etats du `ViewModel` et appeler ses methodes. La signature devient :

```
@Composable
fun ReactionScreen(
    onBackPressed: () -> Unit,
    viewModel: ReactionViewModel = viewModel()
)
```

```
) { ... }
```

3. Navigation avec NavHost

La librairie *Navigation Compose* permet de gerer une pile d'ecrans de facon declarative, sans passer des lambdas de navigation a travers toute l'arborescence.

Les trois elements essentiels :

- **NavController** : l'objet central qui gere la pile de navigation. On le cree une seule fois avec `rememberNavController()`, au niveau le plus haut de l'UI (dans `MiniGamesApp`).
- **NavHost** : le composable qui affiche la destination courante. Il recoit le `NavController` et la liste des destinations possibles.
- **Route** : un objet Kotlin annoté `@Serializable` qui identifie un ecran (ex. `Home`, `Reaction`). Utiliser des objets plutot que des chaines de caracteres permet au compilateur de detecter les erreurs de navigation.

```
// Definition des routes -- objets serializables
@Serializable object Accueil
@Serializable object Detail

@Composable
fun MonApplication() {
    val navController = rememberNavController()

    NavHost(
        navController = navController,
        startDestination = Accueil
    ) {
        composable<Accueil> { _ ->
            EcranAccueil(
                onNaviguer = { navController.navigate(Detail) }
            )
        }
        composable<Detail> { _ ->
            EcranDetail(
                onRetour = { navController.popBackStack() }
            )
        }
    }
}
```

Les routes sont des objets Kotlin annotés `@Serializable` – le compilateur detecte toute erreur de navigation, contrairement aux simples chaines de caracteres. `navigate(Detail)` pousse un nouvel ecran sur la pile. `popBackStack()` retire l'ecran courant et revient au precedent.

3.1. Mise en place de la navigation

Definissez d'abord les trois routes dans un fichier dedie (ex. `Navigation.kt` a la racine du package) :

```
@Serializable object Home
@Serializable object Reaction
@Serializable object WordGame
```

Puis remplacez le if/else de MiniGamesApp par un NavHost :

```
@Composable
fun MiniGamesApp() {
    val navController = rememberNavController()

    NavHost(navController = navController, startDestination = Home) {
        composable<Home> { _ ->
            HomeScreen(
                onReactionClick = { navController.navigate(Reaction) },
                onWordGameClick = { navController.navigate(WordGame) }
            )
        }
        composable<Reaction> { _ ->
            ReactionScreen(onBackClick = { navController.popBackStack() })
        }
        composable<WordGame> { _ ->
            WordGameScreen(onBackClick = { navController.popBackStack() })
        }
    }
}
```

3.2. Adaptation de HomeScreen

HomeScreen doit maintenant proposer deux boutons de jeu. Mettez à jour sa signature :

```
@Composable
fun HomeScreen(
    onReactionClick: () -> Unit,
    onWordGameClick: () -> Unit
) { ... }
```

Ajoutez un deuxième bouton “Mot cache” qui appelle `onWordGameClick`.

4. Mot cache – Trouve le mot dans la grille

4.1. Principe du jeu

Le joueur dispose de **60 secondes** pour trouver le plus grand nombre de mots possible. A chaque manche, une nouvelle grille 3×3 est générée : un mot de 6 lettres est caché parmi 9 lettres mélangées (6 lettres du mot + 3 lettres aléatoires).

S	K	O
L	E	Z
I	L	M

⇒ mot cache : **SOLEIL**

Le joueur constitue son mot en appuyant sur les lettres de la grille une par une. Chaque lettre sélectionnée s’affiche dans une zone de saisie au-dessus de la grille et devient **desactivée** dans la grille. Une icône *effacer* permet de retirer la dernière lettre ajoutée et de la restituer à la grille.

Le jeu se déroule en deux phases :

1. **Phase de jeu** : le timer décompte depuis 60 secondes. Le joueur sélectionne les lettres et valide son mot. Si la réponse est correcte, son score est incrémenté et une nouvelle grille apparaît automatiquement. S’il ne trouve pas, il peut **passer** la grille pour en obtenir une nouvelle sans marquer de point. La partie se termine quand le timer atteint zéro.
2. **Phase de résultats** : le score final (nombre de mots trouvés) est affiché. Le joueur peut rejouer ou revenir à l’accueil.

4.2. WordGameViewModel

```
class WordGameViewModel : ViewModel() {

    enum class Phase { PLAYING, GAME_OVER }

    // Cellule de la grille
    data class Cell(
        val char: Char,
        val isSelected: Boolean = false // desactivee apres selection
    )

    // Vos StateFlow ici (phase, grille, indices selectionnes, score, timer
    ...)

    fun startGame() { ... } // démarre le timer et charge la
    // première grille
    fun selectCell(index: Int) { ... } // ajoute la lettre et desactive la
    // cellule
    fun eraseLast() { ... } // retire la dernière lettre et
    // reactive la cellule
    fun validate() { ... } // si correct : incremente score +
    // nouvelle grille
    fun pass() { ... } // nouvelle grille sans marquer de
    // point
    fun reset() { ... } // retour a PLAYING avec score et
    // timer remis a zero
}
```

La liste de mots peut être déclarée en dur dans le ViewModel (une vingtaine de mots suffit pour le TP) :

```
private val wordList = listOf(
    "SOLEIL", "MAISON", "JARDIN", "CHEMIN", "BOUTON",
    "MIROIR", "PLANTE", "CARTON", "FUSEAU", "CITRON",
    "VIOLON", "RAPIDE", "BLOQUE", "MOUTON", "GATEAU"
)
```

4.3. WordGameScreen

Le composable observe le ViewModel et affiche, de haut en bas :

- Le **timer** : décompte de 60 secondes, affiche en grand.
- Le **score** : nombre de mots trouvés depuis le début de la partie.
- Le **nombre de lettres du mot** à trouver (ex. : “Mot de 6 lettres”), sans révéler le mot lui-même.
- Une **zone de saisie** affichant les lettres sélectionnées au fur et à mesure, accompagnée d’une **icône effacer** qui appelle `eraseLast()`.
- La **grille** 3 × 3 : chaque cellule est un `Button`. Les cellules déjà sélectionnées (`isSelected = true`) sont désactivées (`enabled = false`) et visuellement grises.
- Un bouton **Valider** qui appelle `validate()`, un bouton **Passer** qui appelle `pass()`, et un bouton **Retour** vers l’accueil.

En phase `GAME_OVER`, affichez le score final et proposez de rejouer ou de revenir à l’accueil.

```
@Composable
fun WordGameScreen(
    onBackPressed: () -> Unit,
```



```
viewModel: WordGameViewModel = viewModel()
) {
    LaunchedEffect(Unit) { viewModel.startGame() }
    // ...
}
```

Le `LaunchedEffect(Unit)` garantit qu’une nouvelle partie est lancée à chaque fois que l’écran entre en composition.

5. Ordre d’implémentation

1. **Dependances et synchronisation** : ajoutez les deux dépendances et le plugin de serialisation, puis vérifiez que le projet compile toujours.
2. **ReactionViewModel** : créez le ViewModel et déplacez-y toute la logique du jeu de réaction. Vérifiez que `ReactionScreen` fonctionne toujours à l’identique.
3. **NavHost** : remplacez le `if/else` dans `MiniGamesApp` par un `NavHost`. Adaptez `HomeScreen` avec les deux boutons. Vérifiez la navigation vers `ReactionScreen` et le retour.
4. **WordGameViewModel – logique** : implémentez `startGame()`, `selectCell()`, `eraseLast()`, `validate()` et `pass()`. Gérez le timer avec une coroutine dans `viewModelScope`. Testez en loggant dans Android Studio avant d’avoir l’UI.
5. **WordGameScreen – affichage statique** : construisez la grille 3×3 et la zone de saisie vide, sans interaction pour l’instant.
6. **WordGameScreen – interactions** : branchez `selectCell()` sur chaque cellule, désactivez les cellules sélectionnées, remplissez la zone de saisie et branchez l’icône effacer sur `eraseLast()`.
7. **Phase GAME_OVER** : affichez le score final et gérez la reprise de partie et le retour à l’accueil.
8. **Polish** : soignez les couleurs de sélection, les animations de transition entre les phases, la lisibilité générale.

Bonus

- **Indice** : un bouton “Indice” révèle la première lettre du mot (une seule fois par grille, pénalise le score d’un point).
- **Meilleur score** : affichez le meilleur score obtenu en phase `GAME_OVER` (conserve en mémoire le temps de la session).